From MLP to CNN: Spatial Feature Learning and Classifier-Head Efficiency on BloodMNIST

Victoria Tran (My-Uyen Tran)

Educational Empirical Study

Abstract

This project compares a multilayer perceptron (MLP) and two small convolutional neural networks (CNNs) for eight-class blood cell image classification using BloodMNIST. Three models were trained under the same data splits and fixed optimization settings. Model selection used validation macro-F1, and final performance was measured on the held-out test set. The CNN with a Flatten classifier head achieved the highest test macro-F1 (0.9051), compared with 0.8452 for the CNN using Global Average Pooling (GAP) and 0.8150 for the MLP. GAP reduced trainable parameters by 55.0% relative to Flatten. Results describe this single-seed experiment rather than a general architectural ranking.

1 Introduction and Research Questions

MLPs classify images after flattening their pixels into vectors. CNNs instead use convolutional filters to learn spatial features. This project compares these approaches and examines whether a smaller GAP classifier head can replace Flatten without a large loss in performance.

RQ1 – MLP vs. CNN: How do MLP and CNN architectures compare in classification performance under the same data splits and training settings?

RQ2 – Flatten vs. GAP: How does replacing Flatten with GAP affect trainable parameter counts and classification performance when the CNN feature-extractor design is kept the same?

2 Dataset and Preprocessing

BloodMNIST contains 17,092 RGB images of size 28×28, representing eight blood cell classes. The official splits contain 11,959 training, 1,712 validation, and 3,421 test images. The validation set was used for checkpoint selection; the test set was reserved for final evaluation.

Images were converted to floating-point tensors in $[0, 1]$ using `T oTensor()`. Per-channel mean and population standard deviation were computed from the *training split only*:

$$\boldsymbol{\mu}_{RGB} = (0.794348,\ 0.659659,\ 0.696191), \qquad \boldsymbol{\sigma}_{RGB} = (0.215630,\ 0.241603,\ 0.117889).$$

The same statistics were applied to all splits through $x'_c = (x_c - \mu_c)/\sigma_c$. No resizing or data augmentation was used. Calculating normalization statistics from training images only avoids using validation or test data to define preprocessing.

3 Model Architectures

All models accept input tensors of shape $(B, 3, 28, 28)$ and produce eight raw logits per image. ReLU and Dropout ($p = 0.2$) are used as specified below.

Model	Architecture	Parameters
M0	Flatten → Linear(2352,256) → Linear(256,128) → Linear(128,8); ReLU and Dropout after hidden layers; no BatchNorm	636,296
M1	Two Conv–BatchNorm–ReLU–MaxPool blocks (32, 64 channels) → Flatten → Dropout → Linear(3136,8)	44,680
M2	Same two-block CNN design → GAP → Flatten → Dropout → Linear(64,8)	20,104

M1 and M2 use the same backbone *design*, but were initialized and trained independently. Their main architectural difference is the classifier head. He/Kaiming Uniform initialization was applied to M0 hidden Linear weights and CNN convolutional weights; output Linear layers retained PyTorch's default initialization.

4 Experimental Setup

A single predefined random seed (42) was reset before each independent model run. Training used the Adam optimizer (learning rate 0.001), unweighted cross-entropy loss, batch size 128, and 12 fixed epochs per model. No learning-rate pilot, early stopping, or hyperparameter tuning was performed. Training samples were shuffled using a fresh seed-controlled DataLoader for each run.

After every epoch, validation loss, accuracy, and macro-F1 were recorded. The checkpoint with the highest validation macro-F1 was saved; in a tie, the earlier checkpoint was retained. Each selected checkpoint was then evaluated once on the official test set. Macro-F1 was the primary metric because it gives equal weight to all eight classes.

5 Results

Model	Parameters	Best epoch	Val F1	Test accuracy	Test F1
M0	636,296	12	0.8366	0.8381	0.8150
M1	44,680	7	**0.9156**	**0.9152**	**0.9051**
M2	20,104	11	0.8596	0.8702	0.8452

The ranking by both validation and test macro-F1 was M1 > M2 > M0. Training loss decreased across all three runs. The best checkpoints occurred at different epochs, so the final test evaluations used saved validation-selected weights rather than necessarily the last-epoch weights.

6 Discussion

RQ1 – MLP vs. CNN. M1 exceeded M0 in test macro-F1 by 0.0901 (9.01 percentage points); M2 exceeded M0 by 0.0302 (3.02 points). Both CNNs also used fewer trainable parameters than M0. The results favor the CNN designs for this task under the tested settings. However, M0 and the CNNs differ in BatchNorm and model capacity as well as convolution, so the observed gains cannot be attributed to convolution alone.

RQ2 – Flatten vs. GAP. Flatten provides 3,136 inputs to M1's final classifier, whereas GAP reduces M2's classifier input to 64 values. M2 uses 24,576 fewer parameters than M1, a reduction of 55.0%. Its test macro-F1 was 0.0599 (5.99 points) lower. Thus, M2 offers parameter savings but lower classification performance in this experiment. Parameter reduction is not itself a measurement of training speed or computational cost.

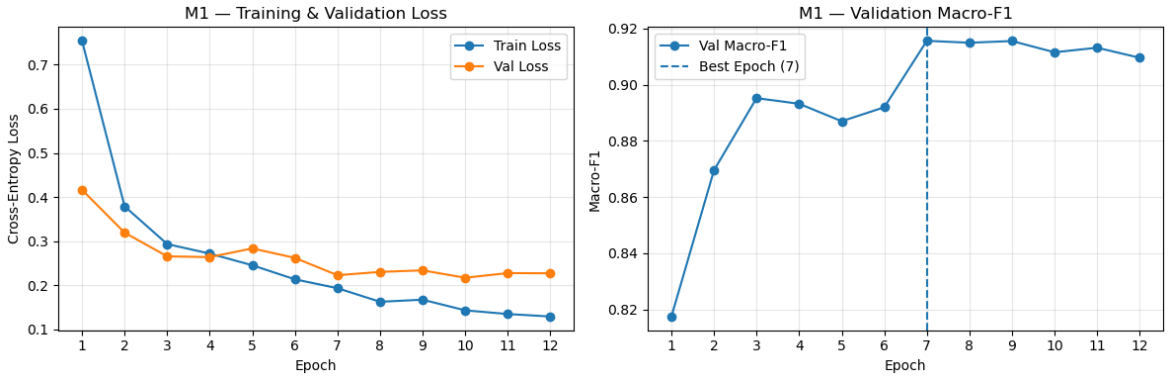


Figure 1: M1 training/validation loss and validation macro-F1 across 12 epochs. The best checkpoint was selected at epoch 7.

7 M2 Error Analysis

The predefined M2 confusion matrix was calculated using the test predictions already obtained during final evaluation. M2 correctly classified 2,977 of 3,421 images and misclassified 444. Monocytes had the lowest recall: 163 of 284 were correctly identified (57.39%). The largest error was monocyte → immature granulocytes (114 images). In contrast, platelet and eosinophil recall were 469/470 and 610/624, respectively.

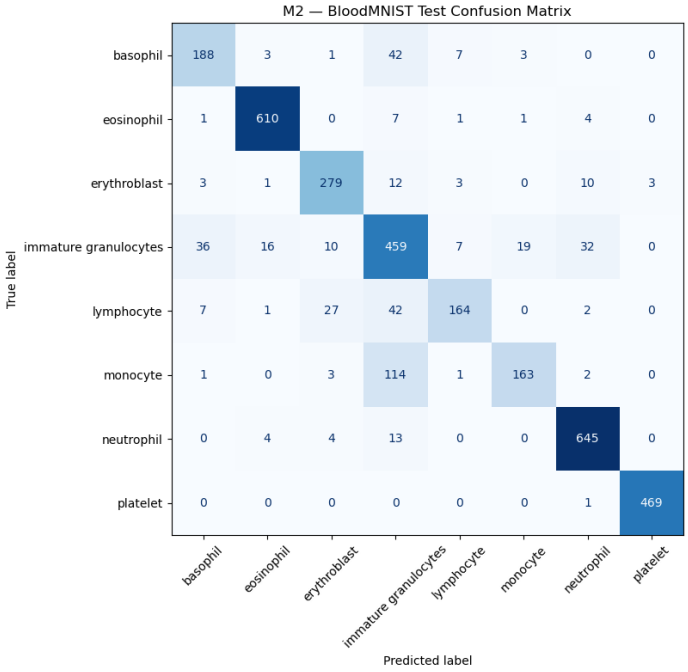


Figure 2: M2 test confusion matrix. Rows are true labels and columns are predicted labels; diagonal entries are correct predictions.

These class-specific differences show why macro-F1 is useful alongside overall accuracy. The confusion matrix describes where errors occur; it does not establish why the model makes them.

8 Limitations

- **Single seed:** Each architecture was trained once, so seed-to-seed variability and statistical significance were not assessed.
- **Architecture differences:** M0 differs from the CNNs in several ways; the experiment does not isolate the causal effect of convolution.
- **Resolution and training settings:** Images are only 28×28; a fixed learning rate and 12-epoch budget may not be optimal for every model.
- **Clinical scope:** Performance on BloodMNIST does not establish reliability on real-world clinical images or suitability for diagnosis.

9 Conclusion

Under this fixed single-seed protocol, both CNNs outperformed the MLP baseline on held-out BloodMNIST test data. M1 achieved the highest test macro-F1 (0.9051). M2 reduced trainable parameters by 55.0% compared with M1 but achieved a lower test macro-F1 (0.8452). The study illustrates how convolutional designs and classifier-head choices relate to classification performance and model size, while remaining limited to the reported experimental setting.

Dataset resource: https://medmnist.com/. **Reproducibility:** See the companion Jupyter notebook, checkpoints, and CSV training histories.